Data Structures

第1章　绪 论

哈尔滨工业大学(威海)计算机科学与技术学院

本章内容提要

1.1　数据结构及其讨论范畴

1.2　基本概念和术语

1.3　抽象数据类型的表示与实现

1.4　算法和算法分析

本章重点难点

重点: ①数据结构的逻辑结构、存储结构以及基本操作的概念及相互关系；②抽象数据类型(ADT)的概念和实现方法；③算法的时间复杂性和空间复杂性分析。

难点: ①抽象数据类型(ADT)的概念和实现方法；②算法的时间复杂性和空间复杂性分析。

本章内容提要

1.1　数据结构及其讨论范畴

1.2　基本概念和术语

1.3　抽象数据类型的表示与实现

1.4　算法和算法分析

◆例：如何在书架上摆放图书？

- 例：如何在书架上摆放图书？
- 方法1：随便放
  - 操作一：新书怎么插入？
  - 操作二：怎么找到某本指定的书？
- 方法2：按照书名的拼音字母顺序摆放
  - 操作一：新书怎么插入？
  - 操作二：怎么找到某本指定的书？二分查找！
- 方法3:把书架划分成几块区域，每块区域指定摆放某种类别的书；在每种类别内，按照书名的拼音字母顺序排放
  - 操作一：先定类别，二分查找确定位置，移出空位
  - 操作二：查找——先定类别，再二分查找

- **解决问题的方法的效率，与数据的组织方式有关。**

实现一个功能的两个程序

◆例：在终端打印从1到N的整数。

◆方法1：

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    for(int i=1;i<n+1;i++)
        printf("%d\t",i);
    return 0;
}
```

◆方法2：

```
#include <stdio.h>
void printN(int N){
    if(N){
        printN(N-1);
        printf("%d\t",N);
    }
}
int main(){
    int n;
    scanf("%d",&n);
    printN(n);
    return 0;   }
```

◆**解决问题的方法的效率，与空间的利用效率有关。**

实现一个功能的两个程序

◆例：求解多项式 $f(x)=a_0+a_2x^2+\ldots+a_{n-1}x^{n-1}+a_nx^n$

```
double f( int n, double a[], double x )
{
    int i;
    double p = a[0];
    for ( i=1; i<=n; i++ )
        p += (a[i] * pow(x, i)) ;
   return p;
}
```

实现一个功能的两个程序

◆例：求解多项式

$$f(x)=a_0 + a_1x + a_2x^2 +\ldots+ a_{n-1}x^{n-1} + a_nx^n$$

$$f(x)=a_0 + x(a_1 + x(\ldots(a_{n-1} + x(a_n))\ldots))$$

```
double f( int n, double a[]，double x )
{
    int i;
    double P = a[n] ;
    for ( i=n; i>0; i-- )
        P = a[i-1] + x*p;
    return P;
}
```

```
p += (a[i] * pow(x, i)) ;
```

◆解决问题的方法的效率，与算法的巧妙程度有关。

1.1 数据结构及其讨论范畴

问题 → 构建数学模型 → 算法实现

分析问题->找出操作对象及其之间的关系

算法 + 数据结构 = 程序设计

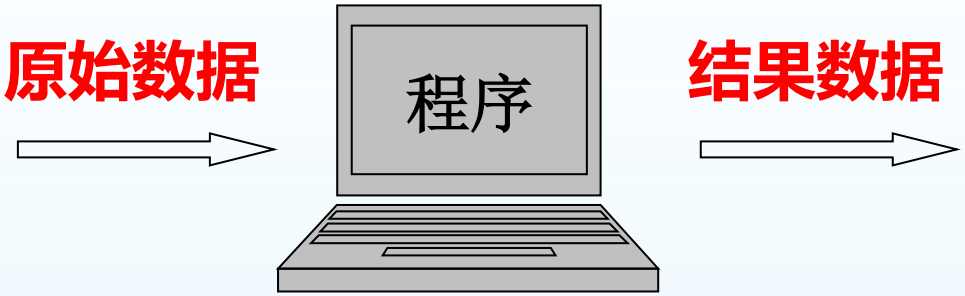
- 处理问题的策略
- 给出问题的数学模型
- 编制出用计算机处理问题的指令

1.1 数据结构及其讨论范畴

数据结构是一门讨论"描述现实世界实体的数学模型(非数值计算)及其上的操作在计算机中如何表示和实现"的学科。

□计算机的发展

软件　硬件　应用领域



□数据处理的种类

数 (整数, 实数)

数值数据

字符　字符串　文字　图形　图象　声音

非数值数据

□数据

对客观对象的符号表示

1.1 数据结构及其讨论范畴

非数值问题

例 已知某级学生情况，要求分班按入学成绩排列顺序。

学号	姓名	性别	出生日期	籍贯	入学成绩	所在班级
00201	杨润生	男	82/06/01	广州	561	00计算机2
00102	石磊	男	83/12/21	汕头	512	00计算机1
00202	李梅	女	83/02/23	阳江	532	00计算机2
00301	马耀先	男	82/07/12	广州	509	00计算机3

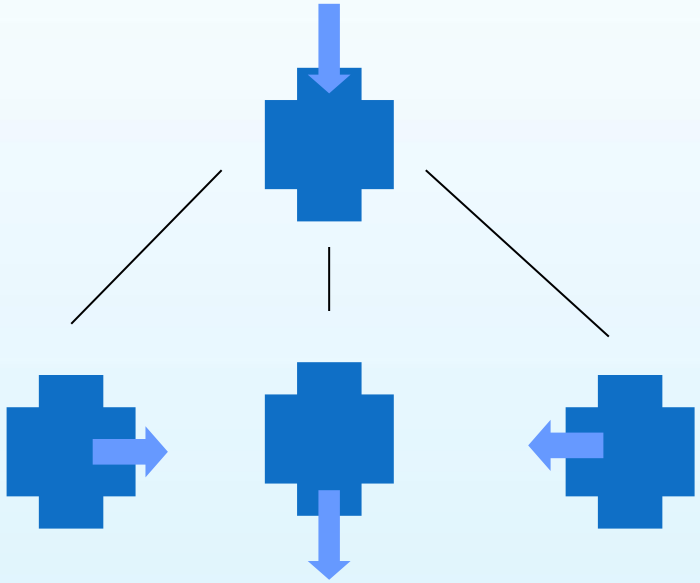
说明：

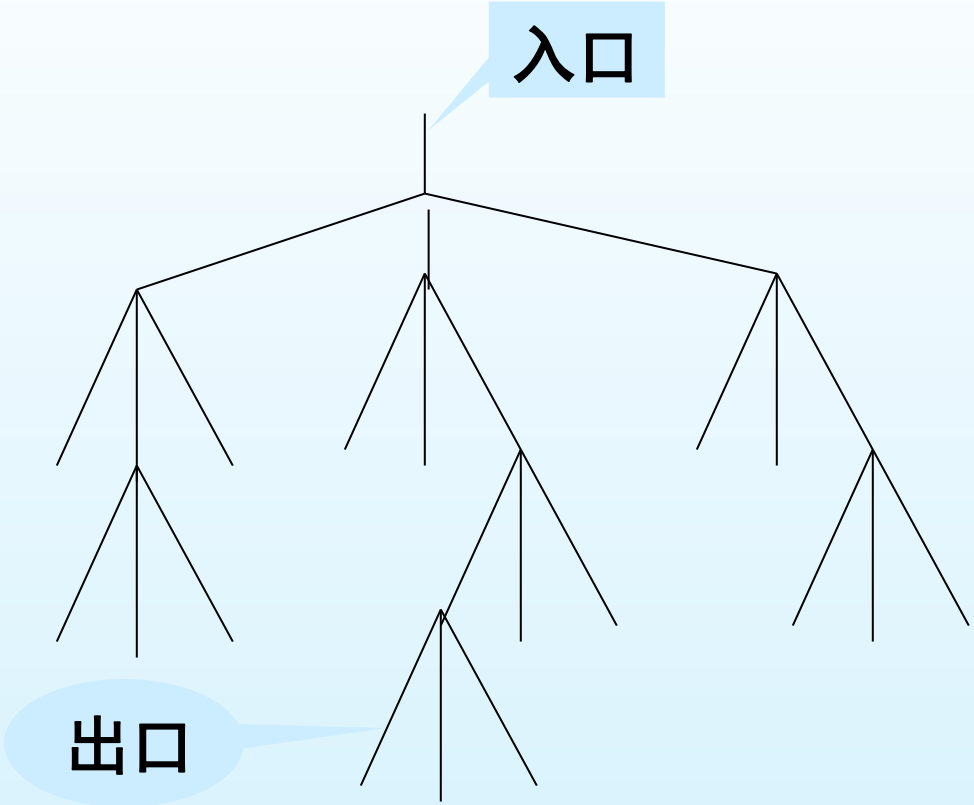
在此类文档管理的数学模型中，计算机处理的对象之间通常存在着一种最简单的线性关系，该数学模型称为**线性模型**。

1.1 数据结构及其讨论范畴

例

迷宫问题：在迷宫中，每走到一处，接下来可走的通路有三条。计算机处理的这类对象之间通常不存在线性关系。若把从迷宫入口处到出口的过程中所有可能的通路都画出，则可得一棵“树”。





1.1 数据结构及其讨论范畴

对每种数据结构，主要讨论如下三方面的问题：

① 数据的逻辑结构

数据元素之间的逻辑关系，是具体关系的抽象。

② 数据的存储结构(物理结构)：

数据元素及其关系在计算机内存中的表示；

③ 数据的运算(基本操作)

即对数据施加的操作。定义在数据的逻辑结构上的抽象的操作。

本章内容提要

1.1　数据结构及其讨论范畴

1.2　基本概念和术语

1.3　抽象数据类型的表示与实现

1.4　算法和算法分析

1.2 基本概念和术语

数据：是信息的载体，能够被计算机识别、存储和加工处理。如整数，实数，字符串、图象、声音等都是数据。

数据元素：数据的基本单位。相当于“记录”,在计算机程序中通常作为一个整体考虑和处理。

数据项: 相当于记录的“域”或字段, 是数据不可分割的最小单位。如学号。

数据对象：性质相同的数据元素的集合。

学号	姓名	性别	出生日期	籍贯	入学成绩	所在班级
00201	杨润生	男	82/06/01	广州	561	00计算机2
00102	石磊	男	83/12/21	汕头	512	00计算机1
00202	李梅	女	83/02/23	阳江	532	00计算机2
00301	马耀先	男	82/07/12	广州	509	00计算机3

1.2 基本概念和术语

数据结构是数据之间的相互关系，即数据的组织形式。

➢**数据结构类型**

- 数据结构
  - 线性结构
    - 线性表
    - 栈
    - 队列
    - 串
    - 数组
    - 广义表
  - 非线性结构
    - 树
    - 图

1.2 基本概念和术语

□数据的存储结构

① 顺序存储方法：数组

② 链接存储方法：指针

③ 索引存储方法

④ 散列存储方法

说明：

- 同一逻辑结构的不同存储结构，冠以不同的数据结构名称。如顺序表、链表、散列表。
- 运算不同，数据结构也不同。如栈和队列。更进一步，顺序栈、链栈、顺序队列、链队列。

单选题 1分

设置

被计算机加工的数据元素不是孤立的,它们彼此之间一般存在某种关系,通常把数据元素之间的这种关系称为( )。

A 结构

B 运算

C 规则

D 集合

提交

本章内容提要

1.1　数据结构及其讨论范畴

1.2　基本概念和术语

1.3　抽象数据类型的表示与实现

1.4　算法和算法分析

1.3 抽象数据类型的表示与实现

□抽象数据型Abstract Data Types（ADT）

[定义]：抽象数据型是一个数学模型和在该模型上定义的操作的集合

例：“矩阵”的抽象数据类型定义

- 类型名称：矩阵（**Matrix**）
- 数据对象集：一个**M * N**的矩阵**A** = (a_{ij}) (i=1, …, M; j=1, …, N)由**M X N**个三元组< a, i, j >构成，其中a是矩阵元素的值，i是元素所在的行号，j是元素所在的列号。
- 操作集：对于任意矩阵**A**、**B**、**C**，以及整数**i**、**j**、**M**、**N**
  - **Matrix Create(int M, int N)**：返回一个**M X N**的空矩阵；
  - **int GetMaxRow(Matrix A)**：返回矩阵**A**的总行数；
  - **int GetMaxCol(Matrix A)**：返回矩阵**A**的总列数；
  - **ElementType GetEntry(Matrix A, int i, int j)**：返回矩阵**A**的第**i**行、第**j**列的元素；
  - **Matrix Add(Matrix A, Matrix B)**：如果**A**和**B**的行、列数一致，则返回矩阵**C=A+B**，否则返回错误标志；
  - **Matrix Multiply(Matrix A, Matrix B)**：如果**A**的列数等于**B**的行数，则返回矩阵**C=AB**，否则返回错误标志；

  ……

1.3 抽象数据类型的表示与实现

□抽象数据类型的规格描述

完整性：反映所定义的抽象数据型的全部特征；

统一性：前后协调，不自相矛盾；

通用性：适用于尽量广泛的对象；

不依赖性：不依赖于程序设计语言。

抽象数据类型实质上就是对一个求解问题的形式化描述（与计算机无关），程序员可以在理解基础上实现它。

1.3 抽象数据类型的表示与实现

抽象数据类型的形式描述

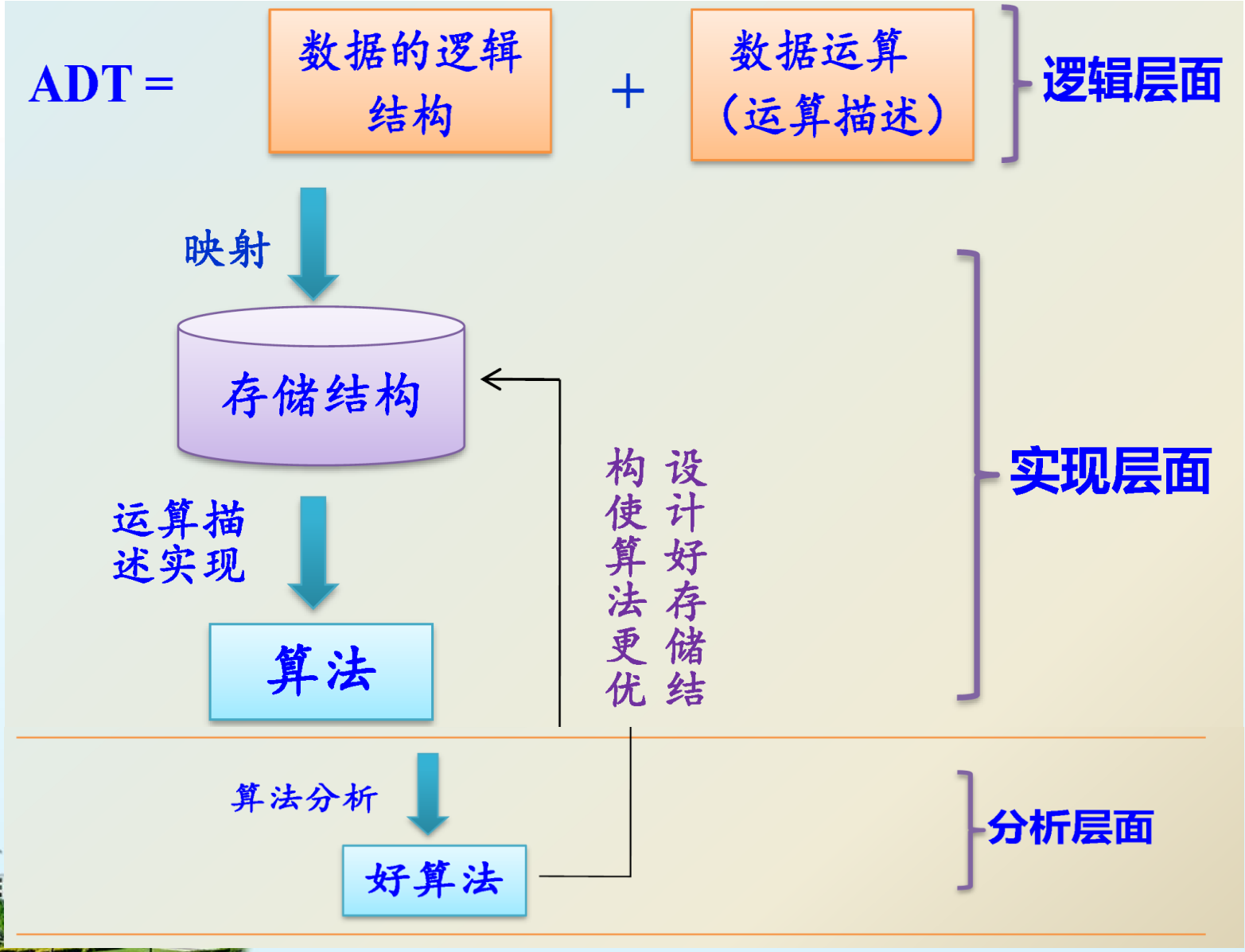
ADT = (D， S， P),其中：

D 是数据对象; S是 D 上的关系集;P是 D 的基本操作集。

数据类型和抽象数据类型

① 抽象数据类型需要通过高级编程语言中已经实现的数据类型（通常称之谓固有数据类型）来实现；

② 抽象数据类型的实现包括数据结构的实现和操作的实现。


ADT =
数据的逻辑结构
+
数据运算（运算描述）
逻辑层面
映射
存储结构
运算描述实现
算法
设计好存储结构使算法更优
实现层面
算法分析
好算法
分析层面

本章内容提要

1.1　数据结构及其讨论范畴

1.2　基本概念和术语

1.3　抽象数据类型的表示与实现

1.4　算法和算法分析

1.4 算法和算法分析

□算法（algorithm）的概念

算法是对问题求解过程的一种描述，是为解决一个或一类问题给出的一个确定的、有限长的**操作序列**。

□算法描述

自然语言；
程序设计语言；
类语言。

关于本书采用的类语言描述：C 或 C++

1.4 算法和算法分析

□ 算法的5个基本特征

- **有穷性**：算法中的操作步骤为有限个，且每个步骤都能在有限时间内完成。
- **确定性**：组成算法的操作必须清晰无二义性。
- **可行性**：算法中的所有操作都必须足够基本，都可以通过已经实现的基本操作运算有限次实现之。
- **输入**：作为算法加工对象的量值，常体现为算法中的一组变量。
- **输出**：它是一组与输入有确定关系的量值，是算法进行信息加工后得到的结果，这种确定关系即为算法的功能。

单选题 1分

设置

考虑下列描述，不能满足算法的特性，试问违反了哪个特性？

```
void exam1()
{   int  n=2;
    while (n%2==0)
        n=n+2;
    printf("%d\n",n);
}
```

A 有穷性

B 确定性

C 可行性

D 正确性

提交

1.4 算法和算法分析

□ 在设计算法时通常应考虑以下原则

- 算法必须是“正确的”
  - 所谓算法是正确的，除了应该满足算法说明中写明的“功能”之外，应对各组典型的带有苛刻条件的输入数据得出正确的结果。
- 应有很好的“可读性”
  - 在算法是正确的前提下，算法的可读性是摆在第一位的，这在当今大型软件需要多人合作完成的环境下是重要的，另一方面，晦涩难读的程序易于隐藏错误而难以调试。

1.4 算法和算法分析

□ 在设计算法时通常应考虑以下原则

- **必须具有“健壮性”**
  - 算法的健壮性指的是，算法应对非法输入的数据作出恰当反映或进行相应处理，一般情况下，应向调用它的函数返回一个表示错误或错误性质的值。
- **算法的效率**
  - 应考虑所设计的算法具有“高效率与低存储量”。高效率与低存储量是矛盾的，要视具体问题而定。

1.4 算法和算法分析

□描述算法的书写规则

- 所有算法均以函数形式给出，算法的输入数据来自参数表；
- 参数表的参数在算法之前均进行类型说明；
- 有关结点结构的类型定义,以及全局变量的说明等均在算法之前进行说明

```
Void bubble_sort(int a[]，int n){
   for (i=n-1; change=TURE; i>=1 && change; --i)
            change=false;
            for (j=0; j<i; ++j)
                if (a[j]>a[j+1])
                    {    a[j] ←→a[j+1] ;   change=TURE ; }
   }
```

算法分析

分析算法占用的资源
- CPU时间 → 时间性能分析
- 内存空间 → 空间性能分析

算法分析目的：分析算法的时空效率以便改进算法性能。

算法时间复杂度分析

一个算法是由控制结构（顺序、分支和循环三种）和原操作构成的。算法执行时间取决于两者的综合效果。

一个算法的基本构成：

控制语句1 原操作	控制语句2 原操作	…	控制语句n 原操作

算法时间复杂度分析

```
void fun(int a[],int n)
{    int i;
     for (i=0;i<n;i++)
          a[i]=2*i;
     for (i=0;i<n;i++)
          printf("%d", a[i]);
     printf("\n");
}
```

原操作

[定义] 语句频度：语句重复执行的次数

算法时间复杂度分析

◆算法分析方式

❶事后分析统计方法：

在比较算法时，把算法首先编程实现，然后计算不同算法的实际运行时间。

一个用高级程序语言编写的程序在计算机上运行时所消耗的时间取决于下列因素：

①依据算法选用的何种策略；

②问题的规模

③所用语言

④编译程序产生的机器代码的质量

⑤机器执行指令的速度

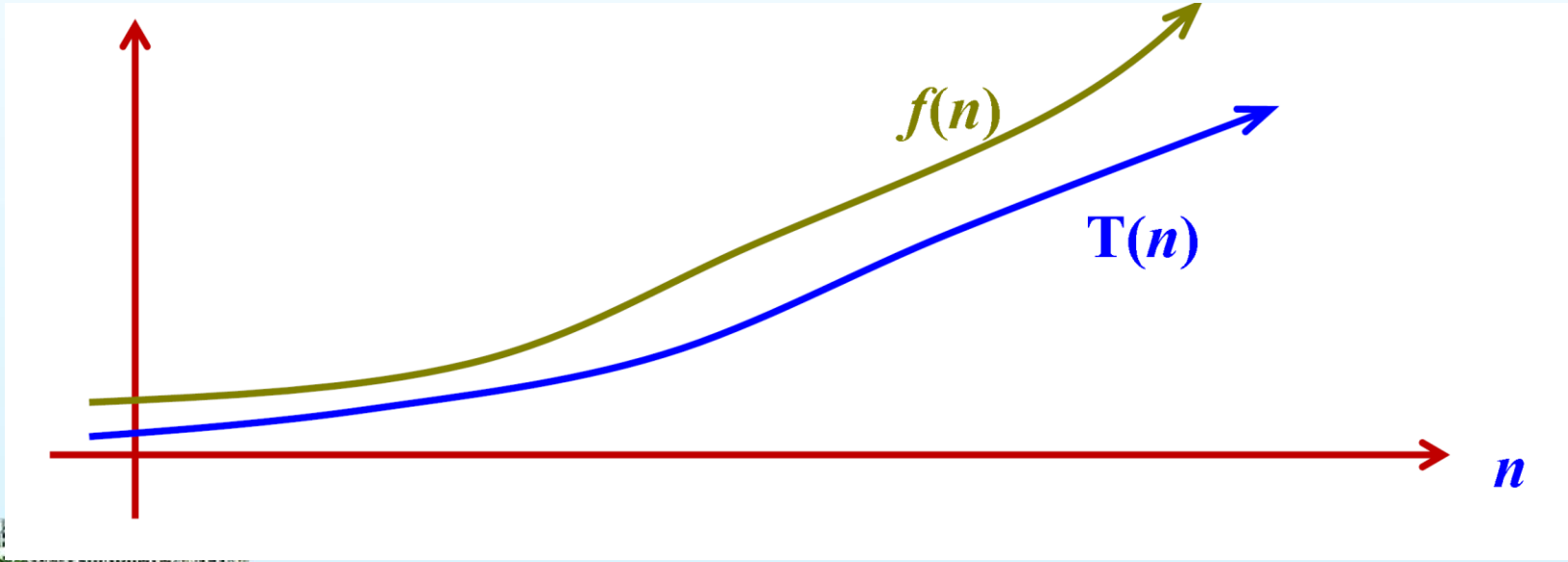
算法时间复杂度分析

◆算法分析方式

❷事前估算分析方法：撇开上述因素，认为算法的执行时间是问题规模*n*的函数。

◆算法中执行时间T(*n*)是问题规模*n*的某个函数*f*(*n*)，记作：T(*n*)=O(*f*(*n*))

➢ 算法所有原操作的执行次数（也称为频度）



算法时间复杂度分析

一般地：

- 一个没有循环的算法的执行时间与问题规模n无关，记作O(1)，也称作常数阶。
- 一个只有一重循环的算法的执行时间与问题规模n的增长呈线性增大关系，记作O(n)，也称线性阶。
- 其余常用的算法时间复杂度还有平方阶O(n^2)、立方阶O(n^3)、对数阶O($\log_2 n$)、指数阶O(2^n)等。

1.4 算法和算法分析

例 n 阶矩阵相乘的算法

```
For ( i = 1; i<=n; i++ )
    For (j = 1; j<=n; j++ )  {

        c[ i ][ j ] = 0 ;
        For (k = 1; k<= n; k++ )
            c[ i ][ j ] += a[ i ][ k ] * b[ k ] [ j ]

    }
```

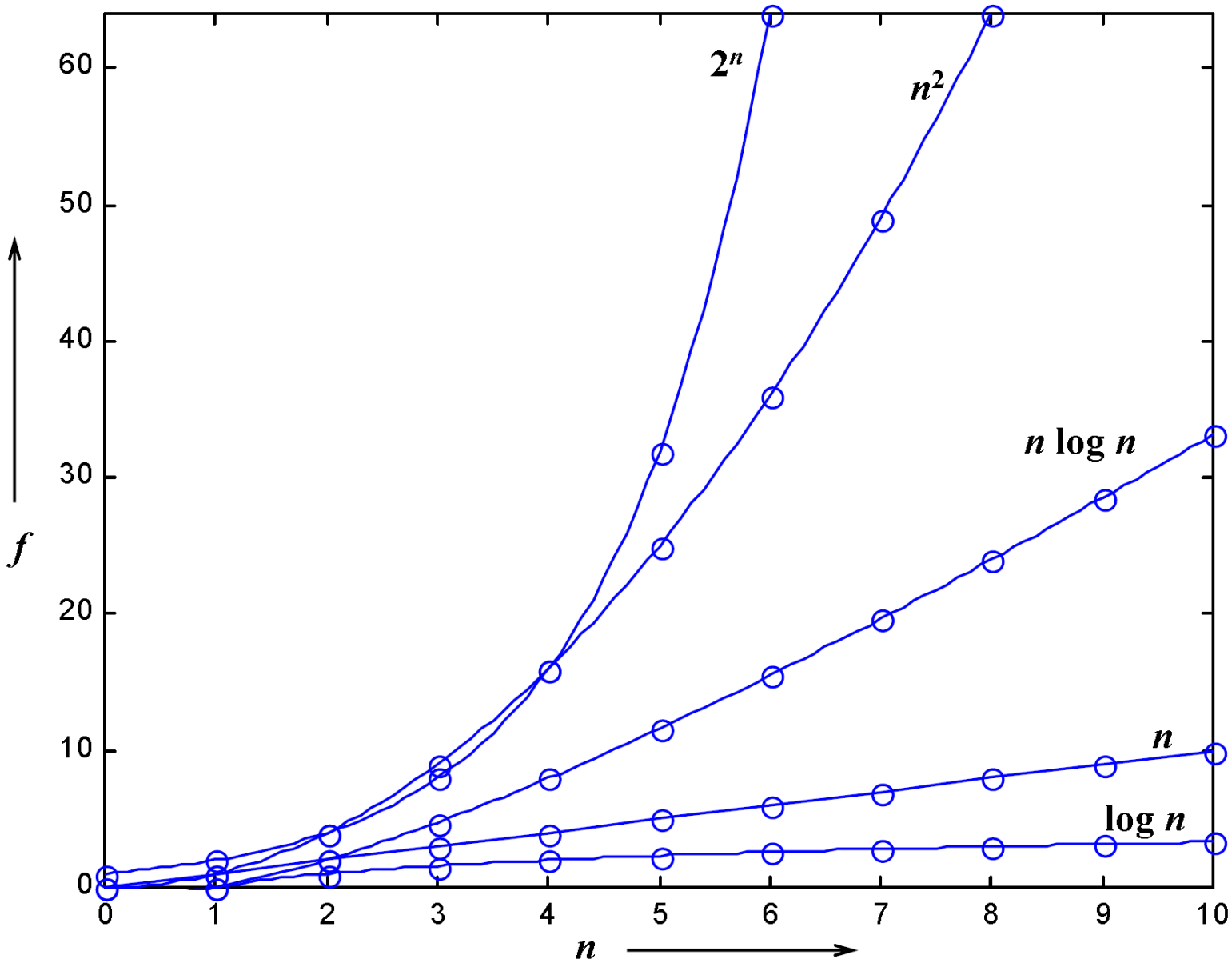
乘法 加法

执行次数均为 n^3

$O(n^3)$ 称为矩阵相乘算法时间复杂度；

$O(n^3)$表示矩阵相乘算法执行时间与n^3同阶；

算法时间复杂性分析方法



1.4 算法和算法分析

□算法的时间复杂度T(n)

- 一般来说，设算法中**基本操作的执行次数**是问题**规模n**的某个函数f(n)
- **算法的时间复杂度**记作： T(n) = O(f(n))
- 表示随问题规模n的增大，算法执行时间的增长率与f(n) 的增长率相同。

说明：有些算法，基本操作执行次数与问题的**输入数据**有关，这时可考虑

① 算法平均时间复杂度

② 算法在最坏情况下的时间复杂度

1.4 算法和算法分析

例 100元买100支笔, 其中钢笔 3元/支, 圆珠笔 2元/支, 铅笔 0.5元/支，各种组合方案的算法如下，计算时间复杂度。

```
#define N 100
void scheme ( ) {
  int i, j, k, count, money;
      for (i = 0;  i<=N; i++ )
            for (j = 0; j<=N; j++ )
                for (k=0; k<=N; k++) {
                    count=i+j+k;
                    money=3*i+2*j+0.5*k;
                    if ( count==N && money==N)
                        printf ("%d, %d, %d \n%", i, j, k) ;
                }
}
```

算法的时间复杂度为O (n^3)

例　　题

例

```
s=0;
    for(i=0;i<n;i++)
      for(j=i;j<n;j++)
        s+=b[i][j];
```

解：最内层语句共运行了：

n+(n-1)+…+1=n(n+1)/2

$O(n^2)$

例　　题

例　下面程序段的时间复杂度为：

```
i=1;
while (i<=n)
    i=i*3;
```

解：设循环了m次

$$3^m=n$$

$$m=\log_3 n$$

$$O(\log_3 n)$$

1.4　算法和算法分析

算法的空间复杂性(Space Complexity)

算法的空间复杂性是指算法在执行过程中的存储量需求；

一个算法的存储量需求除了存放算法本身所有的指令、常数、变量和输入数据外，还包括对数据进行操作的工作单元和存储实现计算所需信息的辅助空间；

一般指辅助空间所占的存储空间。
若不需要辅助空间，则称此算法为原地工作。

本章小结

- ◆数据结构和算法等基本概念；
- ◆数据的逻辑结构：线性结构和非线性结构；
- ◆数据的存储结构可以用以下存储方法；
  - ① 顺序存储
  - ② 链式存储
  - ③ 索引存储
  - ④ 散列存储
- ◆抽象数据类型
- ◆算法的特性
- ◆算法的复杂度分析

思考题

下列程序段的时间复杂度是_____。

```
count=0;
for(k=1;k<=n;k*=2)
    for(j=1;j<=n;j++)
        count++;
```

基本操作

A.$O(\log_2 n)$　　B.$O(n)$　　C.$O(n\log_2 n)$　　D.$O(n^2)$

说明：本题为2014年全国考研题